PROJET 6

ANTICIPEZ LES BESOINS EN CONSOMMATION DE BÂTIMENTS

NICOLAS PETIT | 03/04/2025 | OPENCLASSROOMS

SOMMAIRE

01 Contexte

02 Données à disposition

03 Préparation de la donnée

04 Comparaison des modèles

05 Mise en place du fichier service.py

06 Création de l’image avec Docker

07 Push de l’image docker sur le cloud

01.CONTEXTE

La ville de Seattle se penche sur la consommation des bâtiments non destinés à l'habitation aux alentours des années 2050. Dans le cadre de ces prédictions, l'équipe va récupérer des données, les mettre en forme, et utiliser/comparer des modèles de machine learning afin de trouver le plus adapté au besoin.

De plus, l'image docker personnalisée sera poussée sur une plateforme cloud (aws).

02.DONNÉES À DISPOSITION

Avec les données à disposition :

- Dataset en csv
- 3376 lignes / 46 colonnes
- Caractéristiques sur les bâtiments (type, énergie, surface...)

NB = Utilisation de Kbtu plutôt que Kwh pour donner une dimension plus internationale à ces données.

02.DONNÉES À DISPOSITION

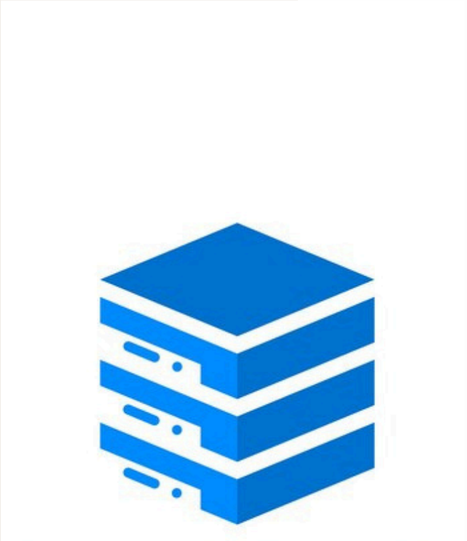
```python
# On regarde le nombre de valeurs manquantes par colonne ainsi que leur type
lines, columns = building_consumption.shape
display(f"Nombre de lignes : {lines}")
display(f"Nombre de colonnes : {columns}")
display("Noms des colonnes :", list(building_consumption.columns))
building_consumption.info()
```

```
'Nombre de lignes : 3376'
'Nombre de colonnes : 46'
'Noms des colonnes :'
['OSEBuildingID',
 'DataYear',
 'BuildingType',
 'PrimaryPropertyType',
 'PropertyName',
 'Address',
 'City',
 'State',
 'ZipCode',
 'TaxParcelIdentificationNumber',
 'CouncilDistrictCode',
 'Neighborhood',
 'Latitude',
 'Longitude',
 'YearBuilt',
 'NumberofBuildings',
 'NumberofFloors',
 'PropertyGFATotal',
 'PropertyGFAParking',
 'PropertyGFABuilding(s)',
 'ListOfAllPropertyUseTypes',
 'LargestPropertyUseType',
 'LargestPropertyUseTypeGFA',
 'SecondLargestPropertyUseType',
 'SecondLargestPropertyUseTypeGFA',
 'ThirdLargestPropertyUseType',
 'ThirdLargestPropertyUseTypeGFA',
 'YearsENERGYSTARCertified',
 'ENERGYSTARScore',
 'SiteEUI(kBtu/sf)',
 'SiteEUIWN(kBtu/sf)',
 'SourceEUI(kBtu/sf)',
 'SourceEUIWN(kBtu/sf)',
 'SiteEnergyUse(kBtu)',
 'SiteEnergyUseWN(kBtu)',
 'SteamUse(kBtu)',
 'Electricity(kWh)',
 'Electricity(kBtu)',
 'NaturalGas(therms)',
```

03.PREPARATION DE LA DONNEE

Avec les données à disposition :

- Choix de la target
- Choix des colonnes à conserver
- Choix des features à créer



NB = Target choisie sur du global et pas par surface pour globaliser les résultats.

03 .PREPARATION DE LA DONNEE

```python
# Création de nouvelles features
building_consumption['BuildingAge'] = 2024 - building_consumption['YearBuilt']
building_consumption['AvgFloorArea'] = building_consumption['PropertyGFATotal'] / building_consumption['NumberofFloors']
building_consumption['ParkingRatio'] = building_consumption['PropertyGFAParking'] / building_consumption['PropertyGFATotal']
```

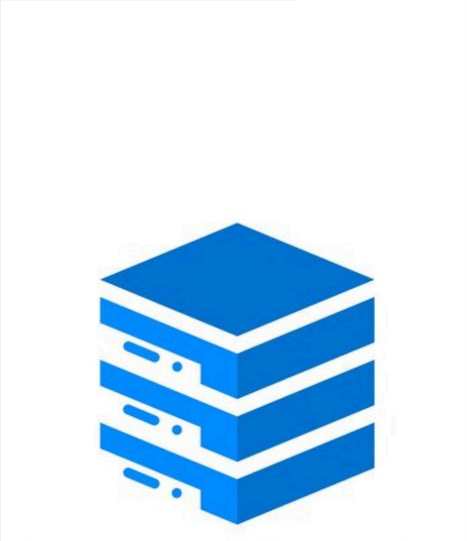
```python
# Suppression des colonnes peu pertinentes
columns_to_keep = [
    'YearBuilt', 'BuildingAge', 'BuildingType', 'NumberofFloors', 'PropertyGFATotal', 'PropertyGFAParking',
    'SiteEnergyUse(kBtu)', 'TotalGHGEmissions', 'Outlier'
]
building_consumption = building_consumption[columns_to_keep]

# Suppression de la colonne YearBuilt
building_consumption = building_consumption.drop(['YearBuilt'], axis=1)
```

04.COMPARAISON DES MODELES

Informations sur les modèles ML :

- Comparaison des modèles (Linear Regression, SVG, RandomForest
- Avec et Sans outliers pour observer les différences
- Tentative d’optimisation de RandomForest
- Choix final du ML
- Sauvegarde du modèle et du scaler



04.COMPARAISON DES MODELES

Résultats AVEC Outliers :

	R^2 Train	R^2 Test	MAE Train	MAE Test
Linear Regression	0.870461	0.850131	1.559938e+06	2.252939e+06
SVR	-0.046996	-0.055599	4.061960e+06	5.942875e+06
Random Forest	0.965279	0.781649	5.529087e+05	2.366457e+06

Résultats SANS Outliers :

	R^2 Train	R^2 Test	MAE Train	MAE Test
Linear Regression	0.746580	0.735757	6.192443e+05	6.362082e+05
SVR	-0.134899	-0.135618	1.386513e+06	1.421549e+06
Random Forest	0.972473	0.802142	1.816507e+05	4.730487e+05

'\n Résultats après optimisation :'

	R^2 Train	R^2 Test	MAE Train	MAE Test
Linear Regression	0.746580	0.735757	6.192443e+05	6.362082e+05
SVR	-0.134899	-0.135618	1.386513e+06	1.421549e+06
Random Forest (Optimisé)	0.921378	0.806172	3.157126e+05	4.650297e+05

04.COMPARAISON DES MODELES

```python
# Sauvegarde du modèle de ML
import bentoml
import pickle

linear_reg_model = LinearRegression()
linear_reg_model.fit(x_train_no_outliers, y_train_no_outliers)

# Sauvegarde avec Pickle
model_path = "linear_regression_model.pkl"
with open(model_path, "wb") as model_file:
    pickle.dump(linear_reg_model, model_file)

print(f"Modèle sauvegardé sous : {model_path}")

saved_model = bentoml.sklearn.save_model("linear_regression_model", linear_reg_model)

print(f"Modèle enregistré avec BentoML : {saved_model}")
```

```
Modèle sauvegardé sous : linear_regression_model.pkl
Modèle enregistré avec BentoML : Model(tag="linear_regression_model:7eoj53qgooryrxh3")
```

```python
# Test de la sauvegarde du fichier
with open("linear_regression_model.pkl", "rb") as model_file:
    loaded_model = pickle.load(model_file)

print("Modèle de régression linéaire chargé !")
```

```
Modèle de régression linéaire chargé !
```

```python
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
```

05.MISE EN PLACE DU FICHIER SERVICE.PY

Points clés du fichier pyhthon :

- Récupération des modèles
- Préparation des données
- Prédiction et résultat API

NB = Conversion du string pour le type de bâtiment en entier pour l'interprétation de l'algorithme.

05.MISE EN PLACE DU FICHIER SERVICE.PY

api > service.py > predict

```python
import bentoml
import pandas as pd
import pickle
from pydantic import BaseModel
from bentoml.io import JSON
from sklearn.preprocessing import StandardScaler

model_ref = bentoml.sklearn.get("linear_regression_model")
model = model_ref.load_model()

with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)  # Charger le scaler sauvegardé

# Créer le service BentoML
svc = bentoml.Service("energy_prediction_service", models=[model_ref])

BUILDING_TYPE_MAPPING = {
    "NonResidential": 0,
    "NonResidential COS": 1,
    "NonResidential WA": 2,
}
FEATURE_ORDER = ["BuildingAge", "BuildingType", "NumberofFloors", "PropertyGFATotal", "PropertyGFAParking", "TotalGHGEmissions", "Outlier"]

# Définition du format d'entrée avec Pydantic
class InputData(BaseModel):
    BuildingAge: int
    BuildingType: str
    NumberofFloors: float
    PropertyGFATotal: float
    PropertyGFAParking: float
    TotalGHGEmissions: float

# Définition du endpoint API
@svc.api(input=JSON(pydantic_model=InputData), output=JSON(), route="/api/v1/energy")
def predict(data: InputData):
    input_data = data.dict()
    # Encodage de la variable catégorielle
    input_data['BuildingType'] = BUILDING_TYPE_MAPPING.get(input_data['BuildingType'], 0)
    input_data['Outlier'] = 0
    # Convertir en DataFrame et ordonner les colonnes
    input_df = pd.DataFrame([input_data])
    input_df = input_df[FEATURE_ORDER]
    # Standardiser les données (IMPORTANT)
    input_scaled = scaler.transform(input_df)  # Appliquer le scaler
    input_scaled_df = pd.DataFrame(input_scaled, columns=FEATURE_ORDER)  # Convertir en DataFrame
    # Faire la prédiction avec le modèle
    prediction = model.predict(input_scaled_df)

    return {"predicted_energy_use": prediction[0]}
```

06.CRÉATION DE L'IMAGE AVEC DOCKER

Principaux axes de la partie Docker :

- Création du dockerfile
- Création du bentofile.yml
- Création du requirements.txt

NB = Préparation du push sur la plateforme cloud

06.CRÉATION DE L'IMAGE AVEC DOCKER

api > Dockerfile > ...

```
FROM python:3.9-slim

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
    build-essential \
    libatlas-base-dev \
    && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 3000

CMD ["bentoml", "serve", "energy_prediction_service:lhuckqagjsryrxh3"]
```

06.CRÉATION DE L'IMAGE AVEC DOCKER

api > ! bentofile.yaml

```yaml
service: "service:svc"
labels:
  owner: "user"
  project: "energy-prediction"
include:
  - "*.py"
  - "*.pkl"
  - "requirements.txt"
python:
  packages:
    - bentoml
    - scikit-learn
    - pandas
    - pydantic
docker:
  distro: "debian"
```

06.CRÉATION DE L'IMAGE AVEC DOCKER

api > requirements.txt

```
bentoml==1.4.3
scikit-learn==1.2.2
pandas==1.5.3
pydantic==2.0.0
joblib==1.2.0
```

07.PUSH DE L'IMAGE DOCKER SUR LE CLOUD

A savoir sur la partie cloud :

- Qu'est ce que le cloud
- Choix de la plateforme
- Environnement accepté par la plateforme

07.PUSH DE L'IMAGE DOCKER SUR LE CLOUD

La vue d'ensemble, l'état et l'ensemble des vulnérabilités de l'analyse des images a été déplacés vers la page détaillée des images. Pour y accéder, cliquez sur une balise d'image.

Images (3)

Supprimer | Détails | Scanner | Afficher les commandes Push

Rechercher des artefacts

1

Balise d'image	Type d'artefact	Transmis à	Taille (Mo)	URI de l'image	Digest	Dernière heure d'extraction enregistrée
latest	Image	27 mars 2025, 16:47:57 (UTC+01)	460.27	Copier l'URI	sha256:a320df0761eb4c31f0b41a6d4b5ff3...	27 mars 2025, 16:59:27 (UTC+01)
-	Image	27 mars 2025, 16:15:01 (UTC+01)	460.27	Copier l'URI	sha256:1d16bfa5a6d319734cc6501f891368...	27 mars 2025, 16:29:54 (UTC+01)
-	Image	21 mars 2025, 15:41:44 (UTC+01)	274.37	Copier l'URI	sha256:439e54f1dfcc05c70e1666b699e220...	27 mars 2025, 12:02:04 (UTC+01)

07.PUSH DE L'IMAGE DOCKER SUR LE CLOUD

APICluster

Dernière mise à jour à
03 avril 2025 à 13:45 (UTC+2:00)

Mettre à jour un cluster | Supprimer un cluster

Aperçu du cluster

ARN	Statut	Surveillance CloudWatch	Instances de conteneur enregistrées
arn:aws:ecs:eu-west-3:551823767387:cluster/APICluster	Actif	Valeur par défaut	-

Services

Drainage	Actif
-	1

Tâches

En attente	En cours d'exécution
-	1

Services | Tâches | Infrastructure | Métriques | Tâches planifiées | Configuration | Balises

Services (1) Info

Gérer les identifications | Mettre à jour | Supprimer le service | Créer

Filtrer les types de lancement: Tout type de lancement

Filtrer les types de service: Tout type de service

Filtrer les services par valeur

1

Service name	ARN	Status	Type de service	Deployments and tasks	Last deployment	Task definition
EnergyPrediction	arn:aws:ecs:eu-v	Actif	REPLICA	1/1 tâches en cours d'exécution	Terminé | Afficher	API_Request:1